ONE LANGUAGE

ENTREGA FORMATIVA | DISENO DEL SOFTWARE

Diseno del software

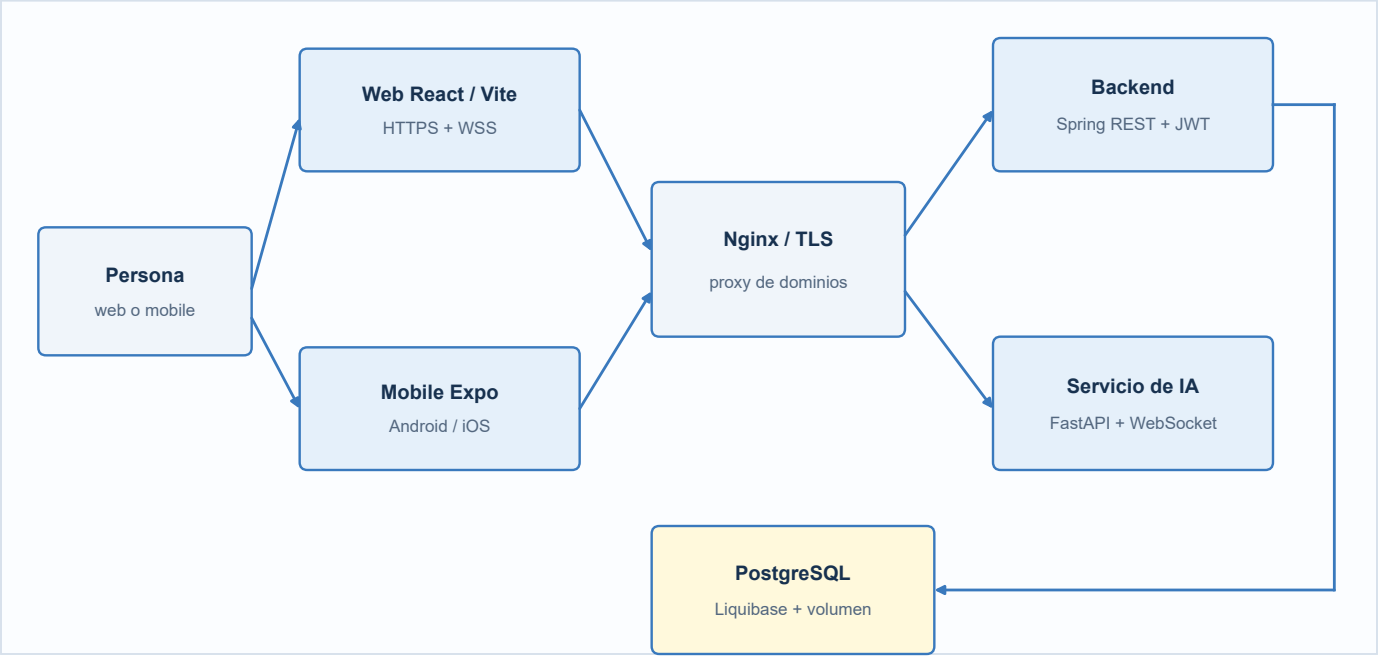
Arquitectura, patrones, interfaces y modelo de datos de la plataforma One Language.

Campo	Detalle
Proyecto	One Language
Version	0.1 | borrador para revision del equipo
Fecha de corte	4 de noviembre de 2026
Alcance	Web, mobile, API, IA y base de datos

Estado documental.
Wireframes esquematicos; requiere revision tecnica del equipo.
No certifica pruebas ni disponibilidad; no incluye secretos.

1. Arquitectura de contexto y contenedores

La arquitectura separa clientes, API transaccional, inferencia de senas y persistencia.
La IA procesa frames; el backend gestiona cuentas e historial y es el unico que accede a PostgreSQL.



Las grabaciones opcionales se guardan en el volumen de medios del backend, fuera de PostgreSQL.

Contenedor	Responsabilidad	Interfaz principal
Web	Sirve la SPA y envia solicitudes al proxy local.	HTTPS; rutas /api y /auth.
Backend	Gestiona identidad, permisos, accesibilidad e historial.	Spring REST en puerto interno 8084.
IA	Recibe frames, calcula landmarks y clasifica secuencias.	WebSocket autenticado; health en 8000.
Base de datos	Persiste datos relacionales y migraciones.	Red interna; acceso desde backend.

Topologia VPS: proyectos Compose separados, Nginx del host y red Docker privada.

2. Secuencia y limites de modulos

Flujo de reconocimiento y guardado

Paso	Origen	Accion y resultado	Destino
1	Persona	Inicia traduccion y concede permiso de camara.	Cliente
2	Cliente	Obtiene token, abre socket y envia auth JWT.	IA
3	IA	Valida origen/token; procesa frames y devuelve prediccion.	Cliente
4	Cliente	Presenta resultado temporal; voz/video son opcionales.	Persona
5	Persona	Confirma guardar o descartar.	Cliente
6	Cliente/API	Envio REST autenticado; backend valida y persiste.	DB + medios
7	API	Devuelve recurso; historial muestra datos propios.	Cliente

Descomposicion logica

Cliente web

Routes/guards -> paginas -> HTTP/socket -> APIs navegador.

Cliente movil

Expo Router -> pantallas -> API/IA -> permisos nativos.

Backend

Controller -> DTO -> Service/transaccion -> Repository/JPA -> DB.

IA

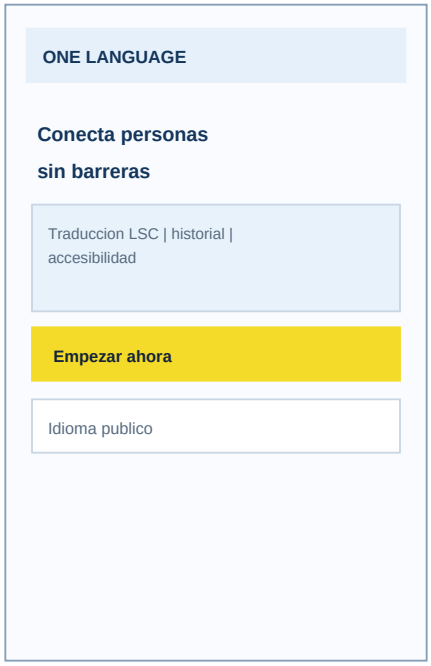
WebSocket -> auth/limites -> landmarks -> clasificador -> JSON.

Limite. El socket no crea por si solo un registro de historial. Persiste cuando el cliente envia explicitamente la traduccion al backend.

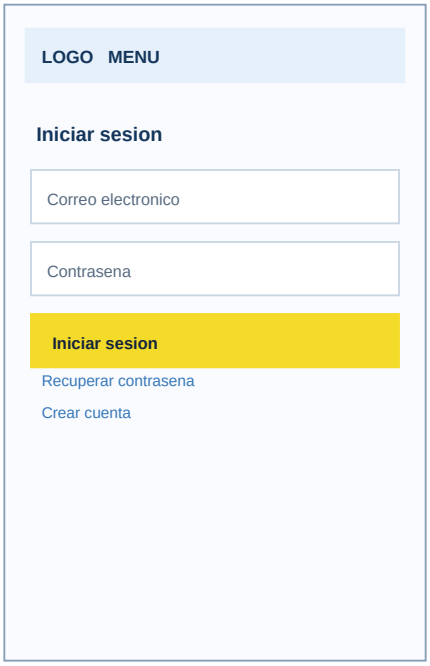
3. Prototipos de interfaces publicas

Wireframes de baja fidelidad: muestran jerarquia, campos y acciones. No son capturas pixel-perfect ni sustituyen pruebas de usabilidad.

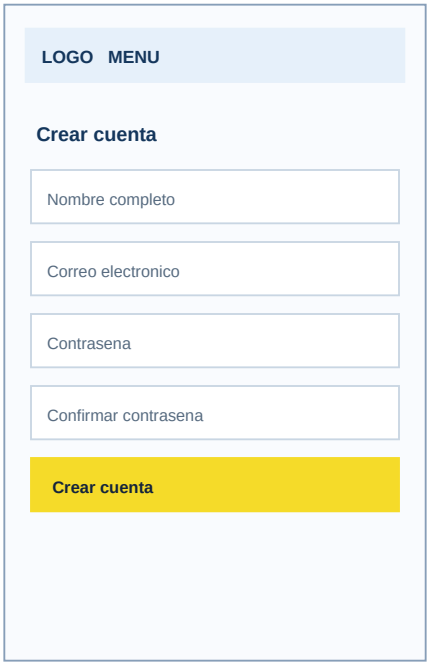
A. Landing publica



B. Inicio de sesion



C. Crear cuenta



Principios de interaccion

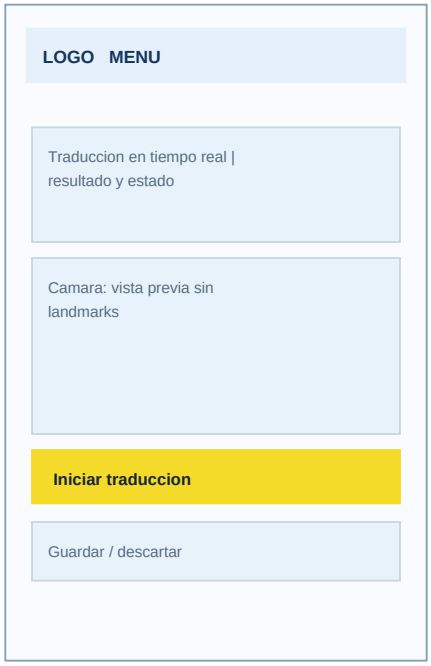
- Rutas publicas separadas de protegidas; una sesion activa no vuelve al acceso.
- Errores junto al campo y accion principal visible.
- Idioma/tema publico se distingue de ajustes autenticados.
- En mobile, teclado no cubre campo activo o boton de envio.

Referencias: AppRoutes.jsx, PublicRoute, ProtectedRoute, PublicLanguageLayout, Login.jsx y Register.jsx.

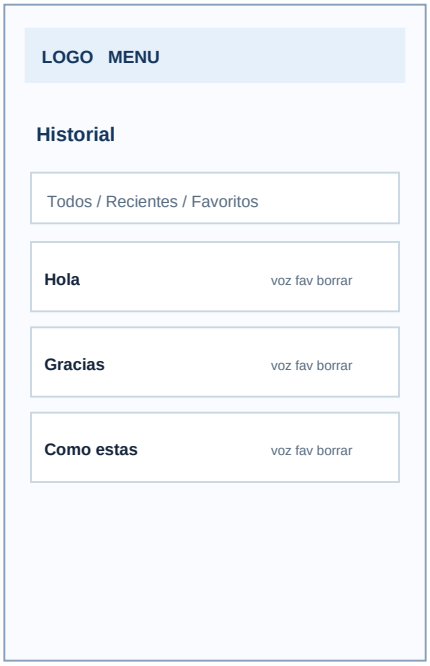
4. Prototipos de interfaces autenticadas

Wireframes de acciones principales; el diseno real responde al viewport, tema y tamano de texto.

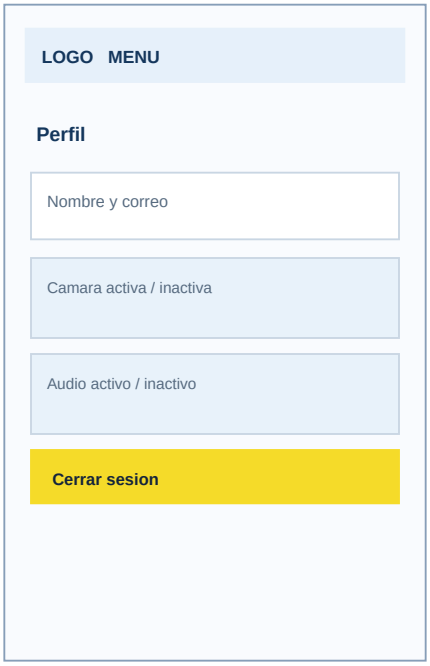
D. Traducir



E. Historial



F. Perfil



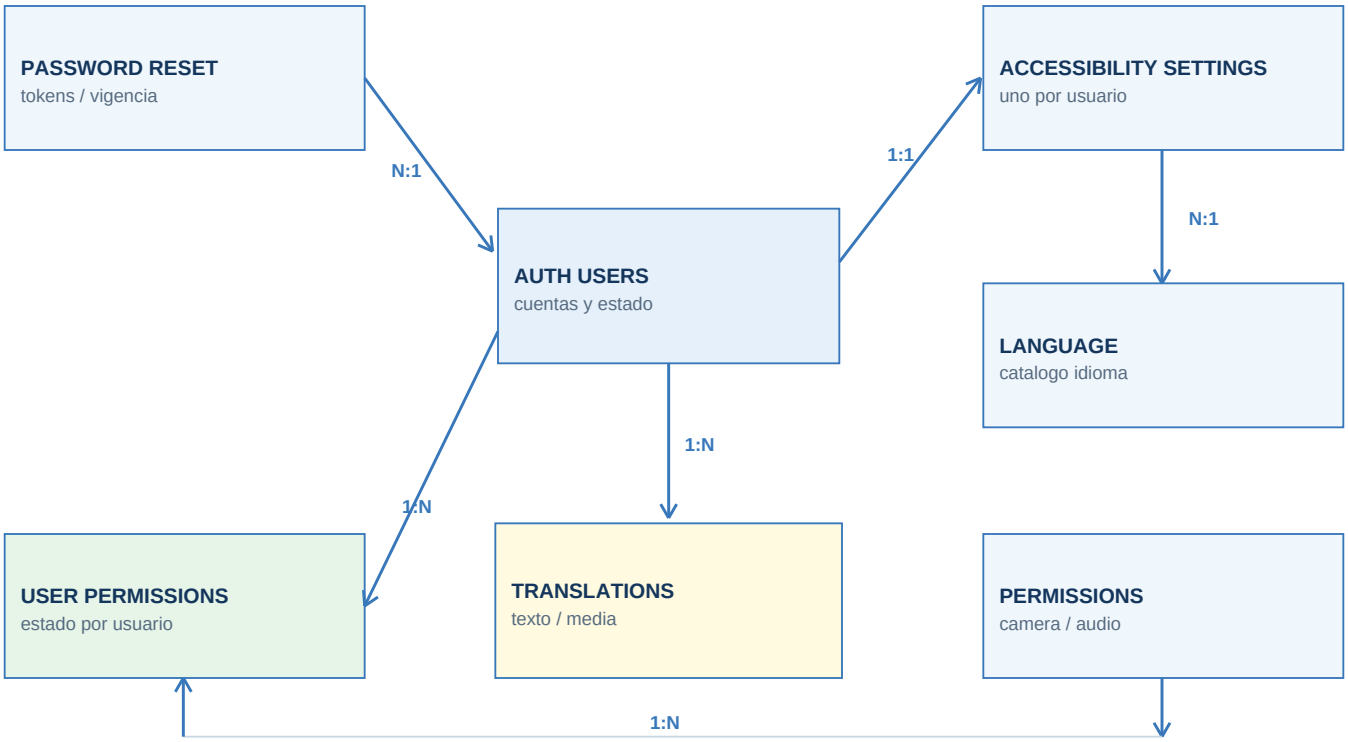
Accesibilidad: texto | tema | idioma

Pantalla	Estados	Comportamiento
Traducir	Permiso, conexion, listo, error, resultado.	No capturar sin permiso; separar red de sin prediccion.
Historial	Cargando, vacio, lista, filtro, borrado.	Acciones por item y aislamiento por cuenta.
Perfil	Datos, permisos, confirmacion salida.	Sincronizar preferencia y permiso del OS.

Accesibilidad. Controles tactiles, textos escalables y reflow mobile deben conservar legibilidad y acciones visibles.

5. Modelo conceptual de datos

Dominios: auth (identidad/autorizacion), translation (resultados) y accessibility (preferencias). No se muestran catalogos historicos que ya no usa el producto.



Relaciones principales

- Un usuario tiene varias traducciones y tokens de recuperacion.
- Un usuario tiene una configuracion de accesibilidad que referencia un idioma.
- Permissions y users se relacionan mediante user_permissions con estado.
- La fila de traduccion guarda metadatos; el binario queda en volumen externo.

Alcance actual. No existe relacion activa con un catalogo de senas. IA devuelve etiquetas; historial guarda texto final.

6. Entidades y esquema relacional vigente

Esquema.tabla	Clave / relaciones	Columnas SQL principales
auth.users	PK id_user; email UNIQUE	UUID; email VARCHAR(255); full_name VARCHAR(150); password_hash TEXT; user_status BOOLEAN; failed_attempts INT; locked_until TIMESTAMPTZ; timestamps y deleted_at.
auth.password_reset_tokens	PK id_token; FK id_user; token_identifier UNIQUE	UUID; token_hash VARCHAR(255); expires_at/used_at TIMESTAMP; timestamps.
auth.permissions	PK id_permission; permission_name UNIQUE	UUID; permission_name VARCHAR(100); description VARCHAR(255); timestamps. Catalogo camera/audio.
auth.user_permissions	PK compuesta (id_user,id_permission); FKs	UUID x2; status VARCHAR(20) con CHECK; granted_at y timestamps.
translation.translations	PK id_translation; FK id_user -> auth.users	UUID; input_type VARCHAR(20); translated_text TEXT; confidence/processing_time FLOAT; status VARCHAR(20); recording_path VARCHAR(255); content_type VARCHAR(100); size BIGINT; timestamps/deleted_at.
accessibility.language	PK id_language; code UNIQUE	UUID; name VARCHAR(50); code VARCHAR(10); is_default/is_active BOOLEAN; timestamps.
accessibility.accessibility_settings	PK id_settings; FK id_user UNIQUE; FK id_language	UUID x2; text_size VARCHAR(50); theme_color VARCHAR(20); voice_feedback BOOLEAN; timestamps.

Reglas de integridad y persistencia

- UUID como identificadores; email unico y requerido; contrasena almacenada como hash.
- Las traducciones usan deleted_at; las tablas dependientes aplican borrado en cascada.
- Grabaciones: la fila guarda ruta, MIME y tamano; el binario vive en almacenamiento de medios.
- Liquibase ordena changesets; PK/UNIQUE crean indices implicitos. El changelog de indices estaba vacio.

Desfase a resolver: voice_feedback existe en SQL, pero no esta mapeado por AccessibilitySettings entity ni DTO/servicio.
Confirmar si debe integrarse o retirarse mediante migracion.

Tablas historicas retiradas: auth.sessions, translation.signs, translation.translation_signs y translation.audio_files.

7. Patrones y estilos observados

Patron	Donde	Uso y limite
Capas / Service Layer	Controller, service, repository y entity por dominio.	Separa HTTP, casos de uso y persistencia; servicio coordina reglas/transacciones.
Repository	TranslationRepository, UserRepository, AccessibilitySettingsRepository.	Encapsula consultas JPA; historial filtra usuario y borrado logico.
DTO + Mapper	DTO request/response; AuthMapper y UserMapper.	Contrato API desacoplado de entidades.
Dependency Injection	Constructor injection de Spring.	Conecta colaboradores y permite sustituirlos en pruebas.
Stateless JWT	AuthServiceImple, JwtService y filtros.	Sin tabla de sesiones; revocacion inmediata necesita mecanismo extra.
Reverse Proxy	Nginx host y Nginx web con rutas API/WSS.	TLS y ruteo; debe preservar upgrade y cabeceras confiables.
Permission association	auth.permissions/user_permissions y FeaturePermissionService.	Estado camera/audio por usuario; no reemplaza permiso del OS.
Composition + hooks	React components/hooks y Expo Router.	UI compuesta con estado; no se observa store Redux global.
Streaming pipeline	IA: WebSocket -> landmarks -> secuencia -> clasificador.	Procesamiento temporal; no persiste por si solo.

No atribuir sin evidencia

No se identifico uso explicito de Factory, Strategy, Observer/Event Bus, CQRS o descubrimiento de servicios. Separar componentes no basta para afirmar esos patrones.

Tradeoffs. JWT facilita escalamiento pero complica revocacion; medios fuera de DB requieren backup y retencion; IA separada aisla dependencias y aumenta operacion.

8. Decisiones, brechas y revision

Decision	Razon	Revision pendiente
IA no conecta a PostgreSQL.	Backend concentra identidad y ownership.	Versionar contrato WS/JSON; medir latencia extremo a extremo.
Guardado explicito.	No almacenar cada prediccion temporal.	Pruebas UX para guardar/descartar y fallos de red.
Permisos de producto y OS separados.	Preferencia de cuenta no concede permiso real.	Probar revocacion/sincronizacion web y mobile.
Medios fuera de fila SQL.	Evita binarios grandes en PostgreSQL.	Definir retencion, borrado y restore coordinado.
Sin catalogo activo de senas.	Producto vigente guarda texto final.	Disenar relacion/versionado futuro solo si cambia alcance.

Criterios de aprobacion

- Confirmar que wireframes representan las ramas de entrega.
- Resolver voice_feedback entre DDL, entity y API.
- Validar claves, tipos e indices contra PostgreSQL de QA.
- Documentar contrato IA y autenticacion antes de cambios incompatibles.
- Validar comprension, controles y reflow con usuarios previstos.

Base tecnica revisada

OneLanguage_Frontend / dev b293cce: rutas, pantallas y clientes. OneLanguage-Backend / dev 6175786: entidades, DTO, controllers, services y repositories. OneLanguage-Database / dev 196c9c1: DDL, migraciones y Liquibase. OneLanguage-AI / develop 781194b: WebSocket y pipeline de landmarks.

Estado. Borrador desde snapshots locales del 4 nov 2026. Requiere revision del equipo; no certifica seguridad ni pruebas de aceptacion.